

Análisis Distribuido de Datos Meteorológicos con MPI

Sistemas Distribuidos - Grupo 2

Integrantes y Roles:

- BEDREGAL PEREZ, DANIEL (Conceptos previos)
- JARA MAMANI, MARIEL ALISSON (Arquitectura del cluster)
- MESTAS ZEGARRA, CHRISTIAN RAUL (Desarrollo y modelamiento)
- NOA CAMINO, YENARO JOEL (Demostración y Dashboard)
- SEQUEIROS CONDORI, LUIS GUSTAVO (Resultados y Discusión)

Introducción y Marco Teórico

¿Qué es MPI (Message Passing Interface)?

- Estándar de comunicación de **memoria distribuida** mediante paso explícito de mensajes.
- **mpi4py**: Librería que adapta la API de MPI al ecosistema científico de Python.

Modelo Maestro-Trabajador (Master-Worker)

- **Maestro**: Orquesta, segmenta la carga del dataset y unifica los resultados.
- **Trabajadores**: Procesan los fragmentos de datos asignados de forma aislada y paralela.
- Ideal para escalar horizontalmente más allá del límite de memoria RAM de un solo servidor.

Arquitectura del Cluster

Infraestructura Virtualizada con Docker Compose

- **Nodos:** Contenedor `master` y contenedores `worker` (escalables horizontalmente).
- **Pilares Técnicos:**
 1. **Comunicación SSH:** Permite a `mpirun` lanzar y sincronizar tareas remotamente.
 2. **Descubrimiento Dinámico:** Resolución de IPs de workers mediante DNS interno (`dig`).
 3. **Optimización con `tmpfs`:** Volumen en memoria RAM (`shm_data`) para transferencias ultrarrápidas de resultados JSON, evitando cuellos de botella de disco.

Operaciones Colectivas en MPI

Distribución y Recolección Eficiente de Datos

- **Scatter:**
 - Divide equitativamente el dataset meteorológico global desde el maestro hacia los workers.
 - Evita enviar duplicados, minimizando la sobrecarga de red del cluster.
- **Gather:**
 - Recolecta y consolida las estructuras de datos de resultados devueltas por cada worker.
 - Permite al maestro realizar la agregación estadística final en memoria de manera centralizada.

Desarrollo de la Solución

Generación de Datos

- Simulación de **1 millón de registros meteorológicos** (temperatura, humedad, viento).
- Incorpora variabilidad sinusoidal para simular estacionalidad y sesgos climáticos geográficos reales.

Procesamiento y Modelamiento

- Cada worker ejecuta análisis estadístico y entrena localmente un modelo de **Regresión Lineal con Scikit-learn** para proyectar tendencias climáticas futuras.

Integración de Visualización

- Los resultados agregados por el maestro se guardan en el volumen RAM compartido.
- Un servidor **Plotly Dash** independiente lee estos archivos directamente de la memoria para renderizar el panel web al instante.

Demostración Práctica (Demo)

Flujo de Ejecución del Cluster

1. Levantar la infraestructura en segundo plano:

```
docker compose up --scale worker=4 -d
```

2. Ejecutar la distribución con MPI a través de la CLI del maestro:

```
./scripts/run_mpi.sh
```

3. Monitorear en consola la distribución con **Scatter** y la agregación final con **Gather**.
4. Visualizar los gráficos actualizados en tiempo real en el **Dashboard** (<http://localhost:8050>).

Resultados del Benchmark

Prueba de rendimiento comparando ejecución secuencial frente a ejecución distribuida con MPI (4 workers) utilizando un dataset de prueba:

Estrategia	Tiempo de Ejecución (s)	Observaciones
Secuencial (1 Proceso)	8.841 s	Optimizado con OpenMP/BLAS por defecto.
MPI Distribuido (4 Workers)	11.166 s	Sobrecarga de comunicación en contenedores.

Justificación de la Sobrecarga:

- Red Virtual en Docker:** Ejecutar contenedores en la misma máquina física añade una sobrecarga de red y CPU que no existiría en un cluster HPC real (con enlaces de baja latencia tipo InfiniBand).
- Paralelismo de Scikit-learn:** Las rutinas internas de Scikit-learn ya aprovechan múltiples núcleos mediante OpenMP sin necesidad de paso de mensajes.

Discusión: Comparativa de Tecnologías

Tecnología	Tipo de Memoria	Escalabilidad	Caso de Uso Ideal
MPI	Distribuida	Alta (Miles de nodos)	Cómputo científico a gran escala y clusters HPC.
OpenMP	Compartida	Limitada a un nodo físico	Paralelismo local multinúcleo de bajo nivel.
CUDA	GPU dedicada	Alta (paralelismo SIMD)	Procesamiento gráfico intensivo y entrenamiento de IA.
Ray	Híbrida / Actor	Alta (Escalado dinámico)	Orquestación moderna de flujos de Machine Learning.

Conclusiones

- **Escalabilidad Horizontal:** Aunque la virtualización en un solo host introduce sobrecargas de red, MPI garantiza que el sistema pueda escalar a múltiples servidores físicos cuando la memoria RAM de un solo nodo sea insuficiente.
- **Arquitectura de Alto Rendimiento:** El uso de Docker Compose con SSH y almacenamiento `tmpfs` emula adecuadamente las condiciones de un cluster de producción real.
- **Integración Moderna:** El proyecto demuestra que es viable conectar el estándar clásico de HPC (MPI) con librerías modernas de Data Science (`mpi4py`, `scikit-learn`, `Plotly Dash`).

¡Muchas Gracias!

¿Preguntas?

Proyecto: Análisis Distribuido de Datos Meteorológicos con MPI

GitHub: <https://github.com/christianmz565/sd-2026-lab-c-grupo-3/tree/main/teo/mpi>